

A Voyage to the Kernel



Part 8

Segment: 2.2, Day 7

We are about to enter the core part of this segment—algorithms. An algorithm could be termed a sequence of computational steps that can transform an input into the output. Here, it should be emphasised that any such sequence cannot be called an algorithm since a wrong methodology will give an incorrect output.

Almost all the code used in this segment will be in a *pseudo-code* format that is akin to C code. Sometimes the algorithms are given in C itself. We have to stick to this format, as our primary intention is to meddle with the kernel.

Those who wish to have an overview of the importance of bringing in innovative algorithms and building simulations (based on them) can look up the GNU Hurd project.

As I promised earlier, this voyage will not neglect the novices. So, let's start with a few simple things. Let's suppose we have a set of values ranging from $x_1, x_2, x_3, \dots, x_n$. If you are asked to arrange them in ascending order, you need to write an algorithm so as to get the output as $x_a, x_b, x_c, \dots$, which satisfies the condition $x_a \leq x_b \leq x_c, \dots$.

This is a simple case of sorting, which is a common algorithm that we employ in our programs.

We just looked at an instance of the sorting problem. This may become more complex depending on the number of items to be

sorted, the extent to which the items are to be sorted, the current state (sorting) of the elements, possible restrictions on the items, and even on the kind of storage device used.

Hence, while dealing with algorithms, we need to consider the data structures employed with which we can manipulate the way to store and organise data in order to facilitate access and modifications as per our requirement.

Finding the shortest route is a kind of sorting algorithm. Consider a trucking company with a central warehouse. Each day, it loads up a truck at the central warehouse and then sends it around to several locations to deliver the products. At the end of each day, the truck should return to the central warehouse so that it is ready to be loaded for the next day. To find out the lowest operating cost, the company needs an algorithm that will indicate a specific order of delivery stops such that the truck travels the lowest overall distance.

If you have enough data in your hands, you can write down an algorithm. Now, let's look at how to write such an algorithm. For the sake of simplicity, let's replace our problem with a simple one.

Consider five cards of clubs—2, 4, 5, 7 and 10—which are placed randomly (just like we had random stops). If you were to play the game, you would have arranged the cards as shown in Figure 1.

But what if a computer had to play your role? How would it arrange the cards? The answer is quite simple: by employing the sorting algorithm. The following code elucidates the algorithm:

```
INSERTION-SORT(X)
for j ← 2 to length(X)
    do key-select ← X[j]
    ▷ Insert X[j] into series X[1... (j-1)]

    i ← j - 1
    while i > 0 and X[i] > key-select
        do X[i + 1] ← X[i]
        i ← i - 1

    X[i + 1] ← key-select
```

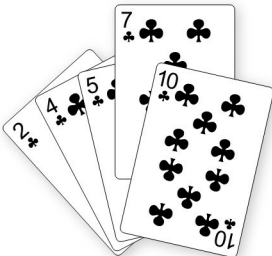


Figure 1: Five cards of clubs arranged in an order

The pseudo code carries elements that are quite akin to those in C, and the alphabets used for each iteration are conventional ones. The character 'j' indicates the 'current card' that is picked up and the ' ' sign symbolises that the remainder of the line is a